Deep Reinforcement Learning

Exploration

Prof. Dr. Sebastian Peitz

Scientific Machine Learning group @ Paderborn University

Winter term 2026/27

PADERBORN UNIVERSITY

Content

Content

- Recap
  - Exploration-exploitation dilemma
  - Multi-armed bandits
- Advanced exploration methods for tabular approaches
  - Upper confidence bounds
  - Thompson sampling
  - Information gain
- Exploration in continuous spaces
  - Novelty-driven exploration: intrinsic motivation
  - The counting problem & pseudo-counts
  - Curiosity-driven exploration
  - Exploration via posterior sampling

- Random network distillation

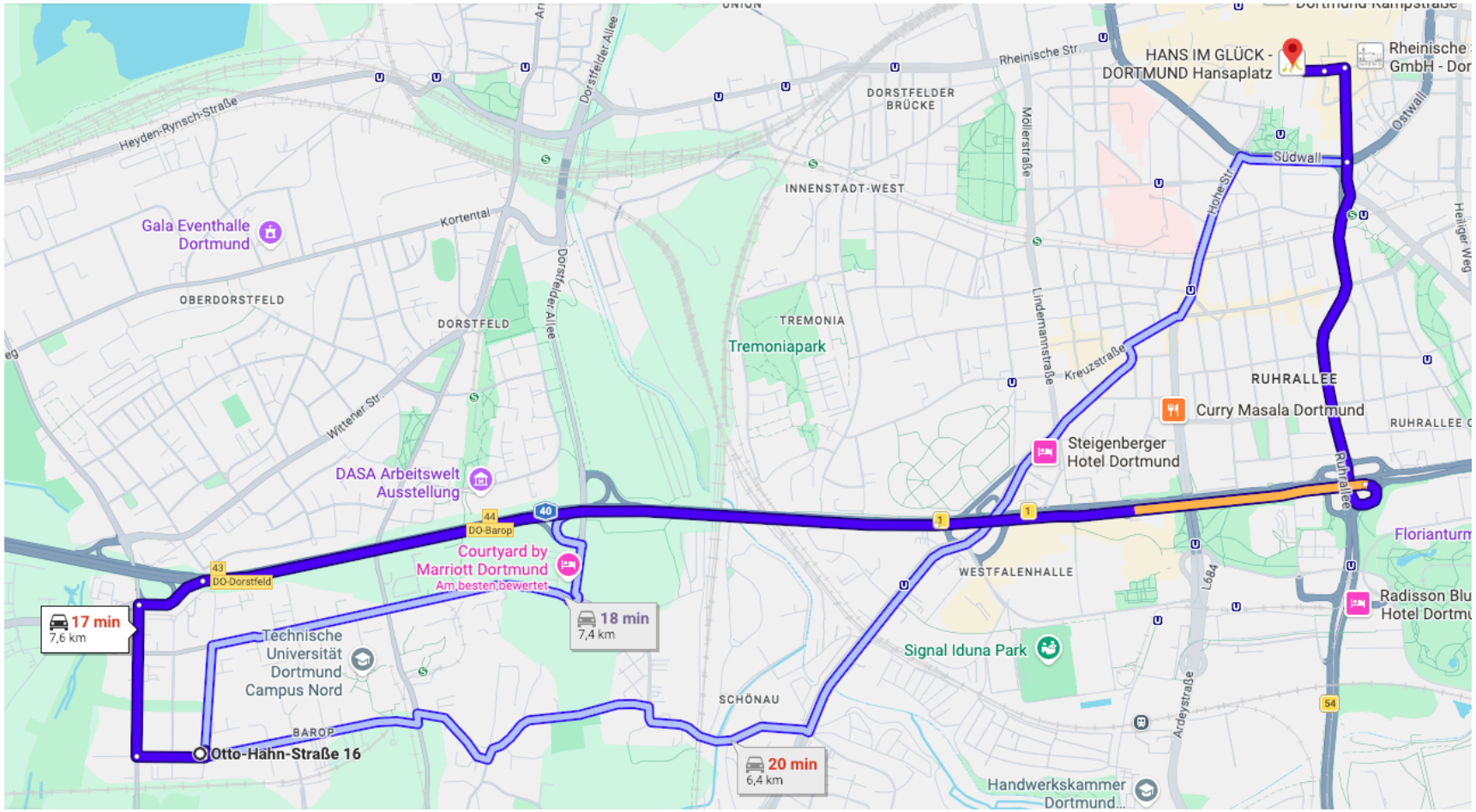
Where are we?

Chapter	Topic	Content
	Basics & tabular methods	
1-5	Bandits, MDPs, Dynamic Programming, Monte Carlo, TD Learning	RL basics in finite dimensions
	Deep-learning-based methods	
6	Brief introduction to deep learning	The basics for what comes next
7	Value function approximation	Value estimation with function approximation
8	Deep Q-learning	Q-learning with neural networks
9	Policy gradients	Direct optimization of the policy
10	Actor-critic algorithms	Improved policy gradients via value functions
11	Advanced algorithms (Part I): From policy gradient to PPO	The PG route to modern RL algorithms
12	Advanced algorithms (Part II): From Q-learning to Soft Actor-Critic	The AC route to modern RL algorithms
13	Exploration	How to explore in complex scenarios
	Model-Based Control	
	Advanced Topics	

Lecture contents

Recap

Exploration-exploitation dilemma



Google Maps (go here for the live version; do you get the same numbers?)

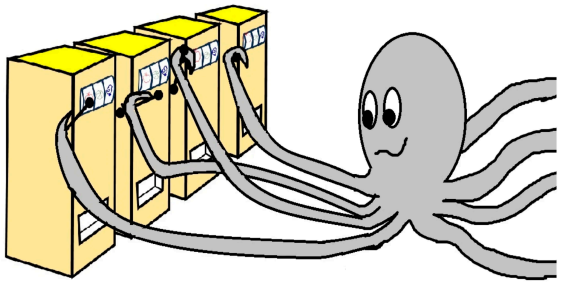
- Let's assume we do not have access to travel time estimates
- Which route should I take to minimize my travel time?
- Let's say we can guess the time of one route fairly well
  - should we always take this one?
  - or try something else and see if we can get better?
- This is known as the **exploration-exploitation dilemma**
- The route pickig problem is one example of a **multi-armed bandit**

Multi-armed bandits

- Let us assume that we have a slot machine and we repeatedly can choose between k different actions
- After each choice a_t you receive a numerical reward r_t chosen from a stationary probability distribution*
- Objective: maximize the **expected total reward** over some time period (e.g., over 1000 action selections, or *time steps*)
- We refer to this as the **value**:

$$q(a) = \mathbb{E}\left[r_t | a_t = a\right]$$

- If we knew $q(a)$, then it would be easy to choose!
- Instead, we have to rely on estimates $Q_t(a)$ which we can iteratively update based on past experience



A multi-armed bandit (Ferreira, Schubert, and Scotti 2024)

A simple bandit algorithm

```
# Initialization
for i in range(k):
  Q(a) = 0
  N(a) = 0

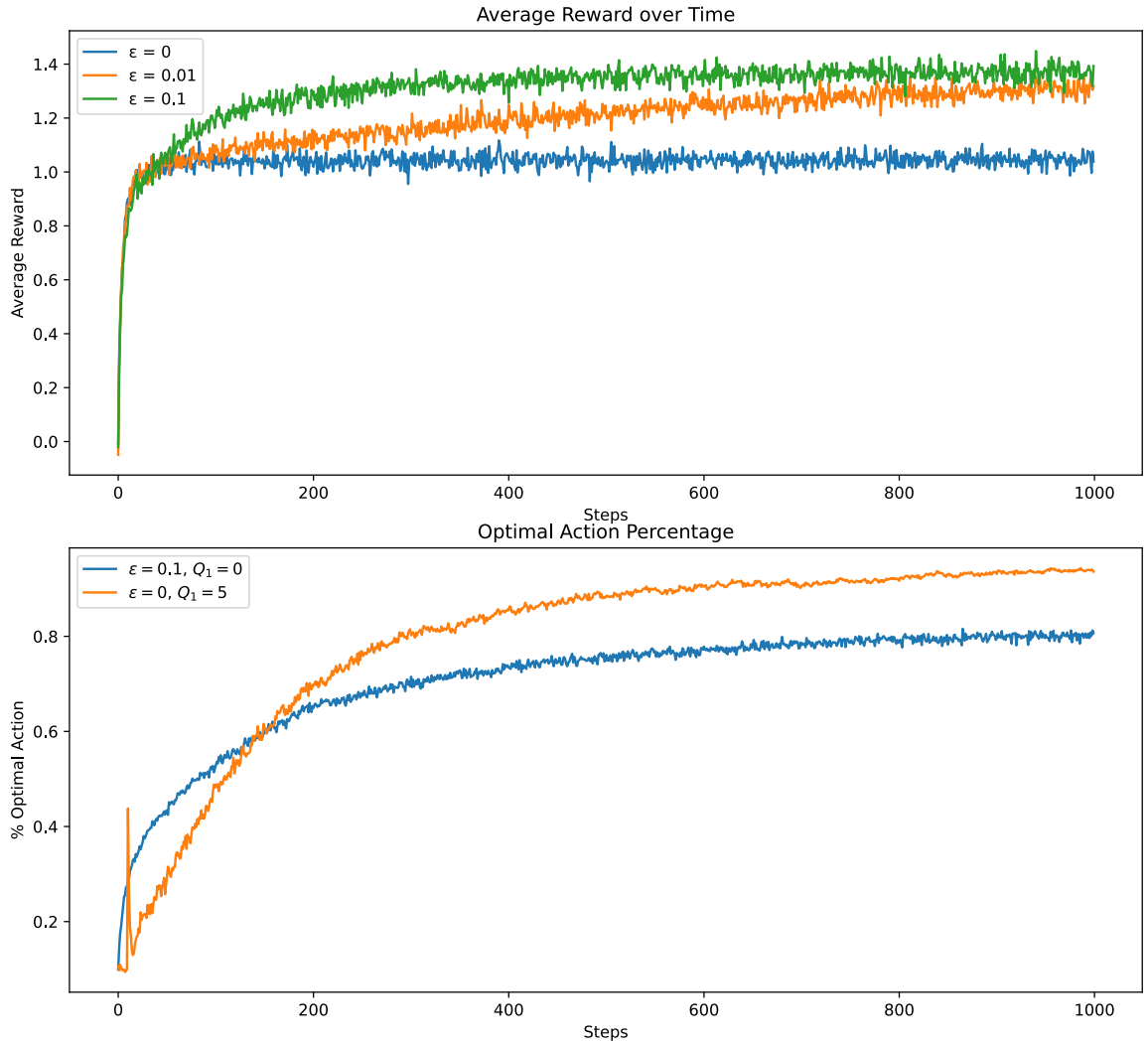
# Run forever
while(True):

  # exploration versus exploitation
  if rand() > epsilon:
    a = argmax(Q)          # exploitation
  else:
    a = randint(k)         # exploration

  r = bandit(a)

  # Error correction towards target r
  N(a) = N(a) + 1
  Q(a) = Q(a) + (1/N(a)) * (r - Q(a))
```

A simple version of the ϵ greedy bandit.



- ϵ-greey is a very simple approach to achieve (local) exploration!
- Another approach: **optimistic initial values**.
  - For $q(a) \sim \mathcal{N}(1, 0)$, an initial guess of $Q_1(a) = 5$ is certainly unrealistically high.

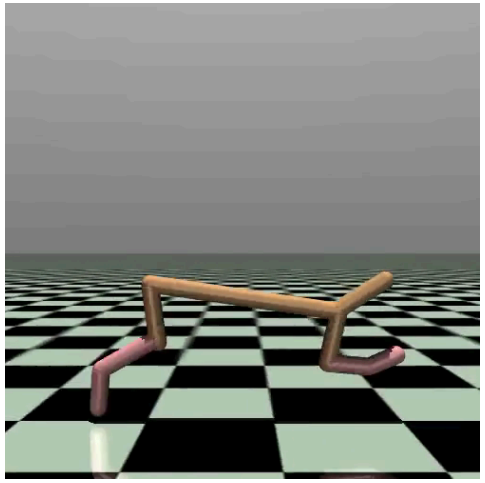
Exploration in Soft Actor Critic

$$L_\pi(\phi) = \sum_{t=0}^{T-1} \gamma^t \mathbb{E}_{(s_t,a_t)\sim\rho_{\pi_\phi}} \left[r_t + \alpha \mathcal{H}(\pi_\phi(\cdot|s_t)) \right] \text{ with entropy } \mathcal{H}(\pi_\phi(\cdot|s)) = \mathbb{E}_{a\sim\pi_\phi(\cdot|s)} \left[-\log \pi_\phi(a|s) \right]$$

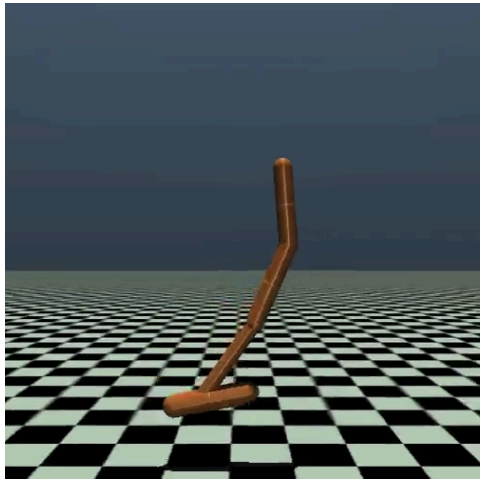
Algorithm 1 Soft Actor-Critic

Input: θ_1, θ_2, ϕ ▷ Initial parameters
$\bar{\theta}_1 \leftarrow \theta_1, \bar{\theta}_2 \leftarrow \theta_2$ ▷ Initialize target network weights
$\mathcal{D} \leftarrow \emptyset$ ▷ Initialize an empty replay pool
for each iteration **do**
  for each environment step **do**
    $\mathbf{a}_t \sim \pi_\phi(\mathbf{a}_t|\mathbf{s}_t)$ ▷ Sample action from the policy
    $\mathbf{s}_{t+1} \sim p(\mathbf{s}_{t+1}|\mathbf{s}_t, \mathbf{a}_t)$ ▷ Sample transition from the environment
    $\mathcal{D} \leftarrow \mathcal{D} \cup \{(\mathbf{s}_t, \mathbf{a}_t, r(\mathbf{s}_t, \mathbf{a}_t), \mathbf{s}_{t+1})\}$ ▷ Store the transition in the replay pool
  end for
  for each gradient step **do**
    $\theta_i \leftarrow \theta_i - \lambda_Q \nabla_{\theta_i} L_Q(\theta_i)$ for $i \in \{1, 2\}$ ▷ Update the Q-function parameters
    $\phi \leftarrow \phi - \lambda_\pi \nabla_\phi L_\pi(\phi)$ ▷ Update policy weights
    $\alpha \leftarrow \alpha - \lambda \nabla_\alpha L(\alpha)$ ▷ Adjust temperature
    $\bar{\theta}_i \leftarrow \tau\theta_i + (1-\tau)\bar{\theta}_i$ for $i \in \{1, 2\}$ ▷ Update target network weights
  end for
end for
Output: θ_1, θ_2, ϕ ▷ Optimized parameters

(Haarnoja et al. 2018)



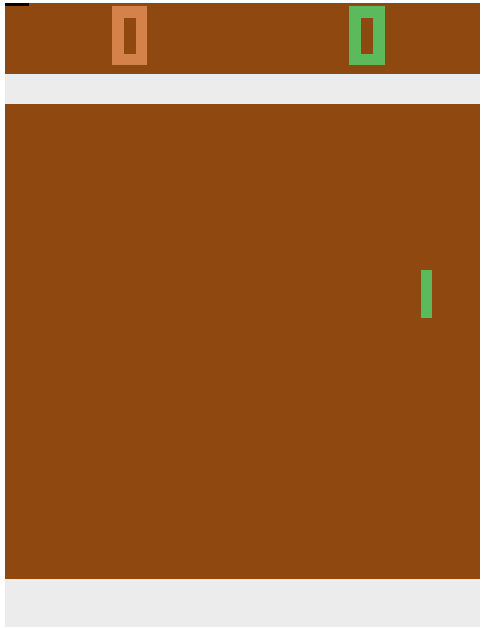
Half cheetah (SAC).



Hopper (SAC).

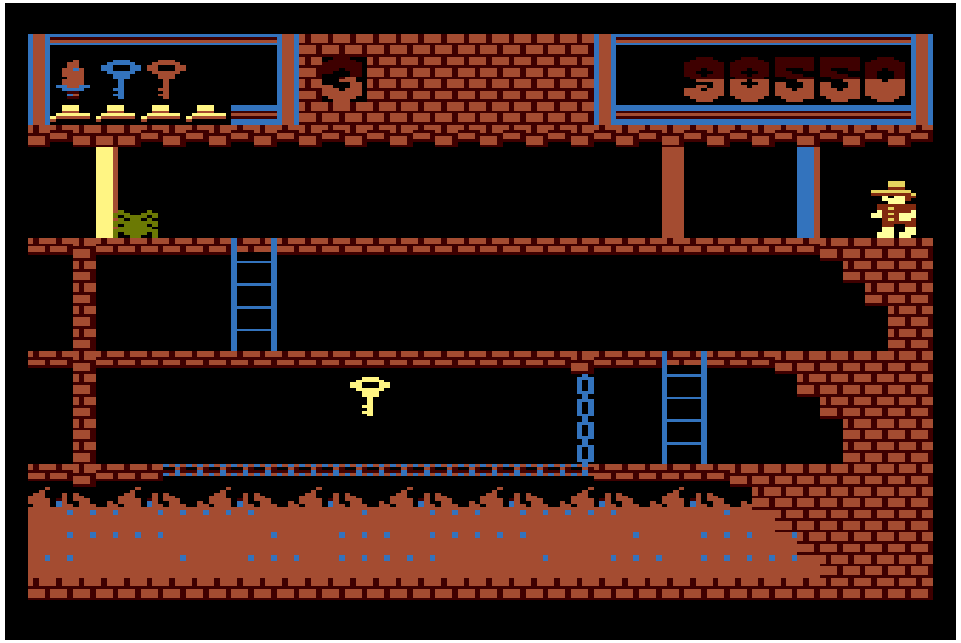
The challenge with exploration

This is “easy”



Pong.

This is very hard!



Montezuma’s revenge.

- Getting key = reward
- Opening door = reward
- Dying = nothing (is it good? bad?)
- Finishing the game only weakly correlates with rewarding events
- We know what to do because we understand the concept!

Another example: The game of Mao



A deck of cards for playing Mao.

- Goal: Get rid of all the cards!
- “The only rule you may be told is this one”.
- There’s a penalty for breaking a rule ⇒ Draw another card.
- You can only discover rules through trial and error.
- The rules don’t always make sense to you.

Temporally extended tasks like Montezuma’s revenge become increasingly difficult based on

- How extended the task is.
- How little you know about the rules.

? Imagine your goal was winning 50 games of Mao in a row, without knowing the rules!

Advanced exploration methods for tabular approaches

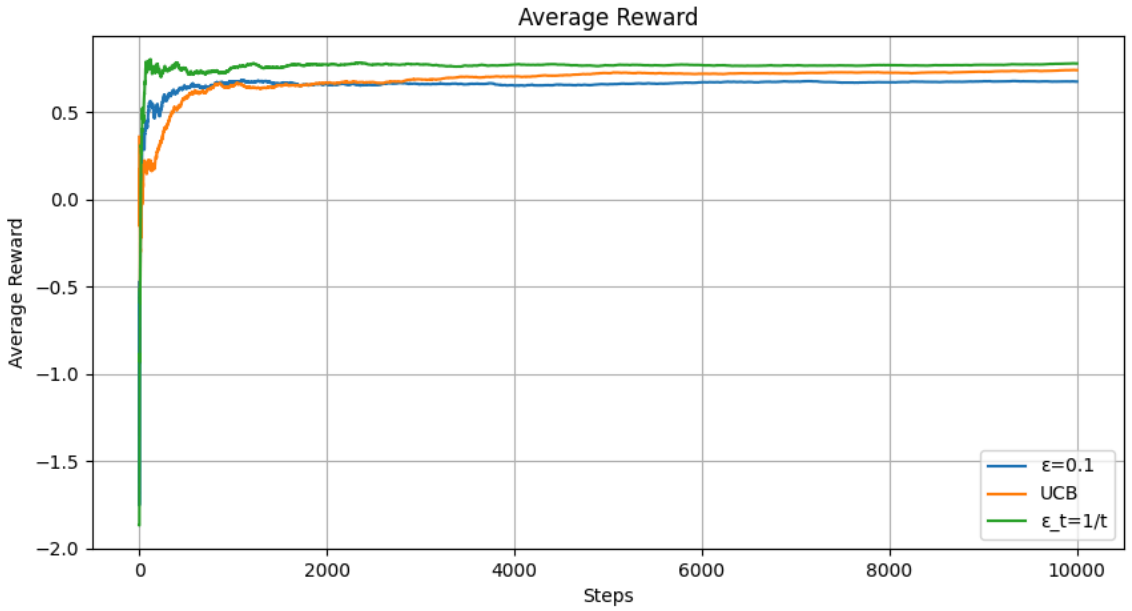
Upper confidence bounds / optimistic exploration

Can we reward explorative behavior in a systematic way?

- Introduce some measure of “curiosity”.
- The less confident we are in an estimate, the higher the chance to get a better reward than we currently expect!
- The technique to include this: **upper confidence bounds** (**UCB**)
  - We add a variance-like term to the reward that estimates the upper end of the confidence interval around the reward.

$$a_t = \arg\max_{a \in \mathcal{A}} \left[Q_{t-1}(a) + c\sqrt{\frac{\ln t}{N_{t-1}(a)}} \right]. \quad (1)$$

  - The more often we have seen an arm (i.e., for larger $N_{t-1}(a)$), the smaller the UCB.



Upper confidence bound in comparison to ε-greedy with constant and scheduled ε.

Bayes' theorem

Bayes' theorem:

$$p(A \cap B) = p(A|B)p(B) = p(B|A)p(A)$$

$$\Leftrightarrow \quad p(B|A) = \frac{p(A|B)p(B)}{p(A)}.$$

$$\text{"Posterior"} = \frac{\text{"Likelihood"} \times \text{"Prior"}}{\text{"Evidence"}}.$$

- Given new **evidence** A,
- we can update our **prior** estimate $p(B)$
- to **posterior** estimate $p(B|A)$
- using the **likelihood** of seeing the new evidence given B.

Example: Medical testing

- Imagine there is a rare disease that only 1% of the population has. There is a test for it which is 90% accurate.
- You take the test and it comes back positive. What are the chances you actually have the disease?
- Consider the two events $A =$ **sick** and $B =$ **positive test**:

$$p(A) = 0.01, \quad p(B|A) = 0.9.$$

- Total probability of the evidence (recall: $p(A) + p(\neg A) = 1$):

$$p(B) = \underbrace{p(B|A)}_{=0.9}\underbrace{p(A)}_{=0.01} + \underbrace{p(B|\neg A)}_{=0.1}\underbrace{p(\neg A)}_{=0.99} = 0.108.$$

- Posterior probability (prob. of having the disease given a positive test):

$$p(A|B) = \frac{p(B|A)p(A)}{p(B)} = \frac{0.9 \cdot 0.01}{0.108} = 0.0833 = 8.33\%.$$

Thompson sampling

The idea behind **Thompson sampling** (also known as **posterior sampling**) is essentially Bayes' theorem:

- Start with a prior distribution over expected rewards (e.g., a Gaussian $\mathcal{N}\left(\mu_k, \sigma_k^2\right), k = 1, \ldots, K$).
- Draw samples from the prior distribution to determine the next action.
- Update to posterior distribution for the action that was selected and the reward that was observed.

Thompson sampling algorithm

For each step $t = 1, 2, 3, \ldots$

1. **Sample**: For each arm k, draw a random number θ_k from its current posterior distribution: $\theta_k \sim \mathcal{N}\left(\mu_k, \sigma_k^2\right)$.
2. **Act**: Select the arm with the highest sampled value: $a_t = \arg\max(\theta_k)$.
3. **Observe**: Pull that arm, receive reward r_t.
4. **Update**: Calculate the new μ_k and σ_k^2 for that specific arm.

Information gain and information-directed sampling

Information gain: "Which arm will give me the most valuable information to help me make better decisions in the future?"

Entropy and information gain

- Let a^* be the true optimal action.
- Goal: reduce uncertainty about a^*.
- Pulling a and observing r: gain information.
- **Information gain** (or **mutual information**) I: expected *entropy* reduction of a^*:

$$I(a) = \mathcal{H}(a^*) - \mathbb{E}\left[\mathcal{H}(a^*)|r\right].$$

  - $I(a)$: How much pulling arm a will clarify which arm is truly the best.
  - If an arm's reward distribution is highly uncertain, pulling yields a large information gain.
  - If we already know exactly what an arm does, pulling it yields zero information gain.

Information-directed sampling (IDS)

- Maximizing information gain would waste our time testing highly unpredictable arms that we already know have poor rewards.
- We need to consider the immediate loss, which we call **regret** (Δ).
  - The expected regret of pulling arm a is the difference between the maximum possible expected reward and the expected reward of arm a:

$$\Delta(a) = \mathbb{E}[R(a^*)] - \mathbb{E}[R(a)]$$

  - **Information-directed sampling** (**IDS**) defines the information ratio for each arm:

$$\Psi(a) = \frac{\Delta(a)^2}{I(a)}$$

  - "Cost-to-benefit" ratio: regret is the cost, information gain is the benefit.
  - A low ratio means we are either getting a high reward (low regret Δ) or learning a lot about the system (high information gain I), or both.

💡 Using the Gaussian assumption for the rewards and our beliefs, one can compute closed-form expressions for $\Delta(a)$ and $I(a)$.

- IDS objective: $a_t = \mathbf{arg\,min}_{a \in \mathcal{A}} \Psi(a)$.

Summary

- **Optimistic exploration** by estimating upper confidence bounds.
- **Posterior sampling**: updating the belief of a reward (the prior) by using evidence $\Rightarrow$ Bayes' theorem.
- **Information gain**: which actions give me the most information about the world?

Exploration in continuous spaces

Novelty-driven exploration: intrinsic motivation

- As we have seen before (e.g., the UCB in Eq. (1)), a common theme for exploration to add some form of bonus r^i to the reward signal that promotes exploration (also referred to as **intrinsic reward**, thus the superscript ^{i}):

$$\hat{r} = r + r^i(N(s)).$$

- This is a special case of the general concept of **intrinsic motivation**: We modify the reward signal in such a way that the agent is *motivated* (i.e., rewarded) to explore unknown territory:

$$\hat{r} = r + \beta r^i.$$

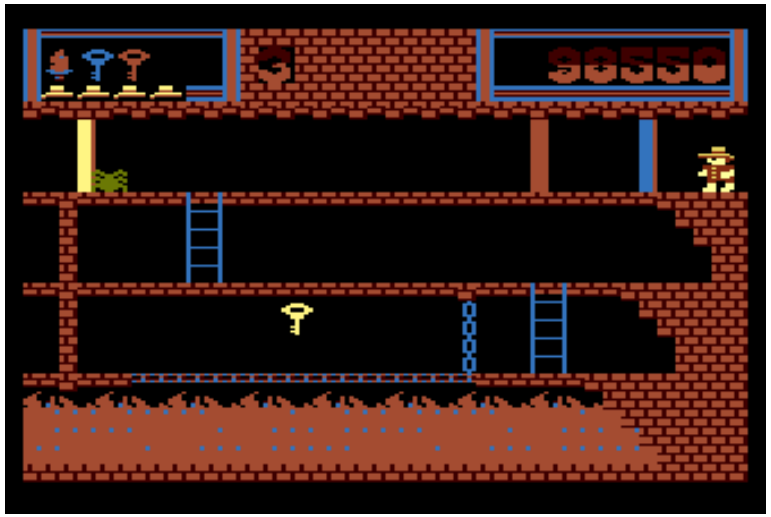
Type of intrinsic motivation	Core question the agent asks	Key algorithms
Novelty / information-theoretic	"Have I seen this specific state configuration before?"	Pseudo-counts (CTS, PixelCNN), count-based hashing
Prediction error / surprise	"Can I predict what happens next if I take this action?"	ICM (intrinsic curiosity module)
Knowledge gain / reduction of uncertainty	"How much does this experience improve my model of the world?"	RND (random network distillation), disagreement ensembles

The counting problem

- As motivated in UCB (Eq. (1)), we can add a bonus B to the reward that shrinks with the number of times we have seem certain state s:

$$\hat{r} = r + r^i(N(s))$$
$$\text{with} \quad s' > s \Rightarrow N(s') < N(s).$$

- But what does counting mean in continuous spaces?

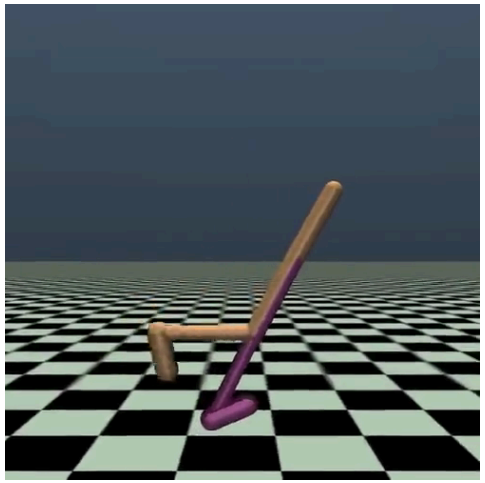


Montezuma's revenge.

- Let's revisit the Montezuma's revenge example:
  - What if the key is placed slightly differently?
  - Or the spider?
  - Or some other minor detail?

The challenge in continuous $\mathcal{S}$

- We "never" see the same thing twice (i.e., with probability zero).
- However, some states are more similar than others.
  $\Rightarrow$ back to function approximation.



Walker 2D (SAC).

Pseudo-counts

- (Bellemare et al. 2016): Fit a density model $p_\theta(s)$ (or $p_\theta(s,a)$).
- $p_\theta(s)$ may be high even for a new s if s is similar to previously seen states.
- For small MDPs, the true (i.e., *tabular*) probability **before** and **after** a state visit is:

$$\text{Before:} \quad \underbrace{p(s)}_{\substack{\text{probability} \\ \text{/ density}}} = \frac{\overbrace{N(s)}^{\text{count}}}{\underbrace{n}_{\text{total visits}}}$$

$$\text{After:} \quad p'(s) = \frac{N(s)+1}{n+1}.$$

- Can we use $p_\theta(s)$ and $p_{\theta'}(s)$ to get a **pseudo-count** $\hat{N}(s)$?
- Bonus used by (Bellemare et al. 2016): $r^i(\hat{N}(s)) = \sqrt{\frac{1}{\hat{N}(s)}}$.
- Various alternatives available.

RL with pseudo-counts

- Fit model $p_\theta(s)$ to all states $\mathcal{D}$ seen so far.
- Take step t and osberve s_t.
- Fit new model $p_{\theta'}(s)$ to $\mathcal{D} \cup s_t$.
- Set $\hat{r}_t = r_t + r^i(\hat{N}(s))$.

Computing $\hat{N}(s)$

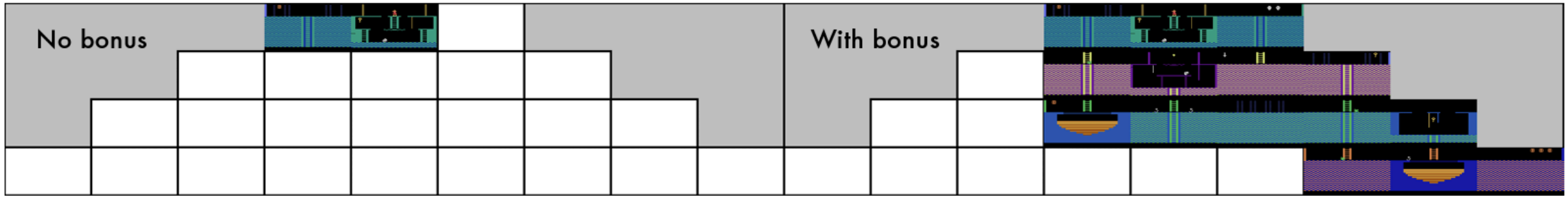
$$p_\theta(s_t) = \frac{\hat{N}(s_t)}{\hat{n}}, \qquad p_{\theta'}(s_t) = \frac{\hat{N}(s_t)+1}{\hat{n}+1}.$$

$\Rightarrow$ Tow equations for two unknowns ($\hat{N}$ and $\hat{n}$)!

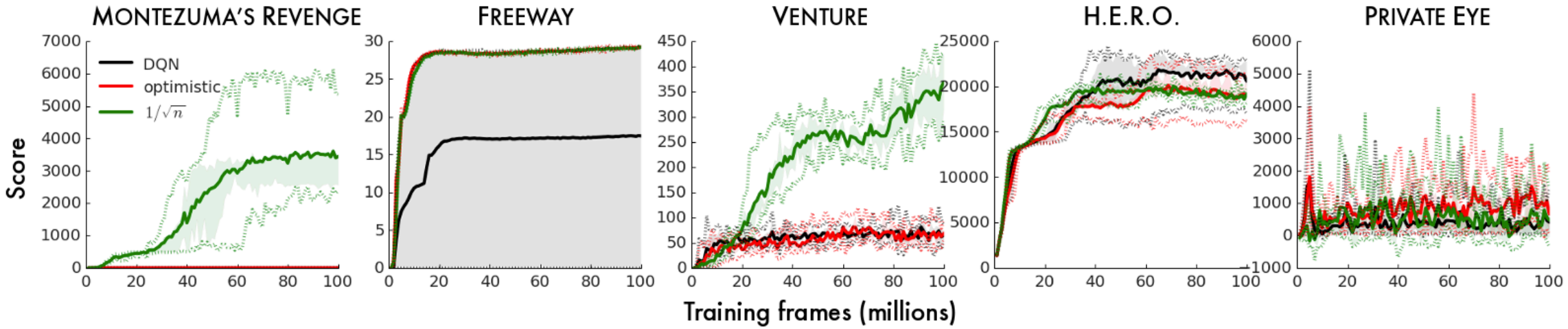
$$\hat{N}(s_t) = \hat{n} p_\theta(s_t), \qquad \hat{n} = \frac{1 - p_{\theta'}(s_t)}{p_{\theta'}(s_t) - p_\theta(s_t)},$$

$$\hat{N}(s_t) = \frac{(1 - p_{\theta'}(s_t)) p_\theta(s_t)}{p_{\theta'}(s_t) - p_\theta(s_t)}.$$

Example: Count-based exploration in Atari games



Exploration of rooms/levels in Montezuma's revenge (Bellemare et al. 2016).



Performance on various Atari games (Bellemare et al. 2016).

Ways to estimate the density

- There are numerous ways to estimate the state density $p_\theta(s)$ or state-action density $p_\theta(s, a)$.
  - Context tree switching (CTS) (Bellemare et al. 2016).
  - Neural networks (e.g., PixelCNN (Ostrovski et al. 2017)).
  - "Exemplar models" (Fu, Co-Reyes, and Levine 2017).
  - Grouping via Hashing (Tang et al. 2017): An autoencoder compresses states into a finite-size latent space (e.g., a binary code). Similar states are grouped together $\Rightarrow$ back to finite $\hat{\mathcal{S}}$!
  - …
- Key takeaways:
  - $p_\theta(s)$ does not need to produce great samples s.
  - It just needs to yield a good approximation of the state density distribution!

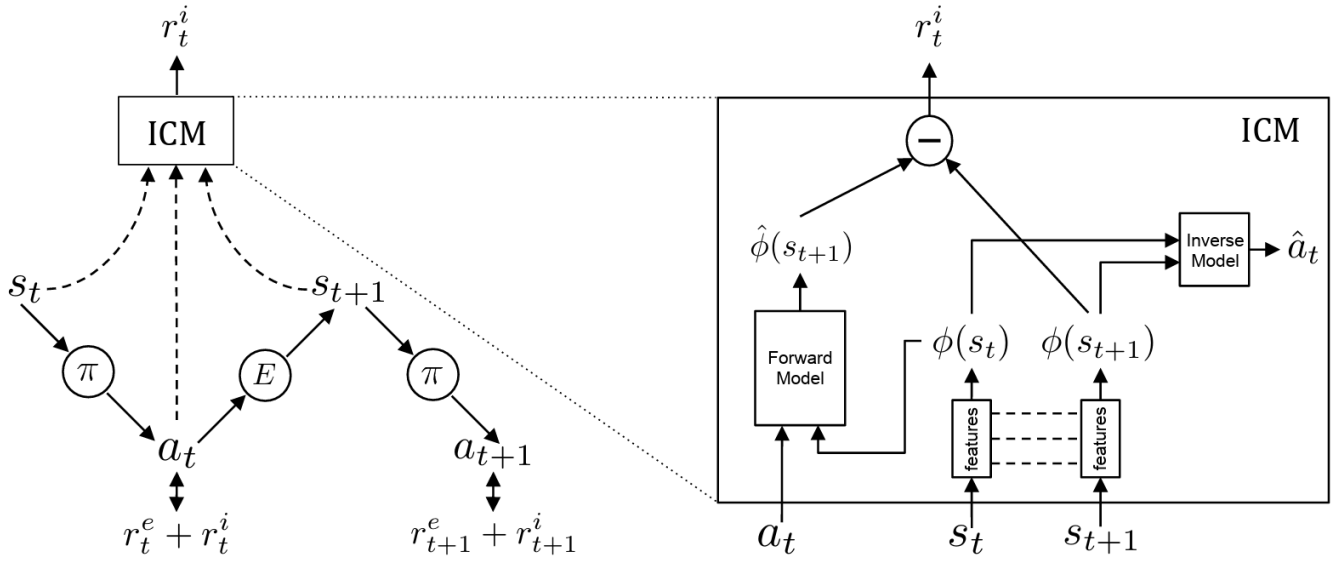
Curiosity-driven exploration

Instead of counting states, an **Intrinsic Curiosity Module** (**ICM**) (Pathak et al. 2017) uses a predictive neural network model:

- The agent in state s_t executes $a \sim \pi \Longrightarrow s_{t+1}$.
- π is trained to maximize the sum of the *extrinsic reward* (r_t^e) by the environment E and the *intrinsic reward* (r_t^i) generated by the ICM:

$$r_t = r_t^e + r_t^i.$$

- ICM: The prediction error in feature space is used as the curiosity based intrinsic reward signal r_t^i.
  - The *forward model* takes as inputs $\phi(s_t)$ and a_t and predicts the feature representation $\hat{\phi}(s_{t+1})$ of s_{t+1}.
  - ICM encodes s_t, s_{t+1} into the features $\phi(s_t)$, $\phi(s_{t+1})$ that are trained to predict a_t (the *inverse dynamics model*).

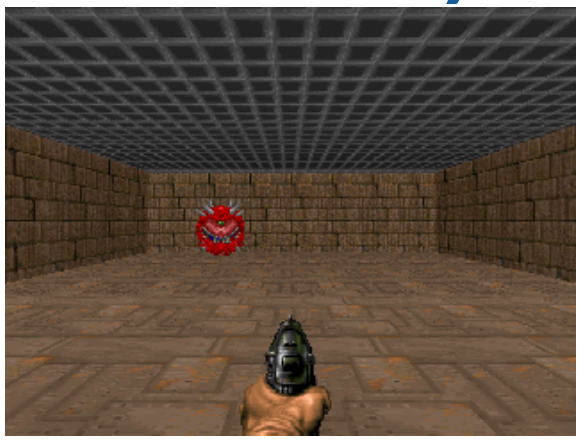


Intrinsic Curiosity Module (Pathak et al. 2017).

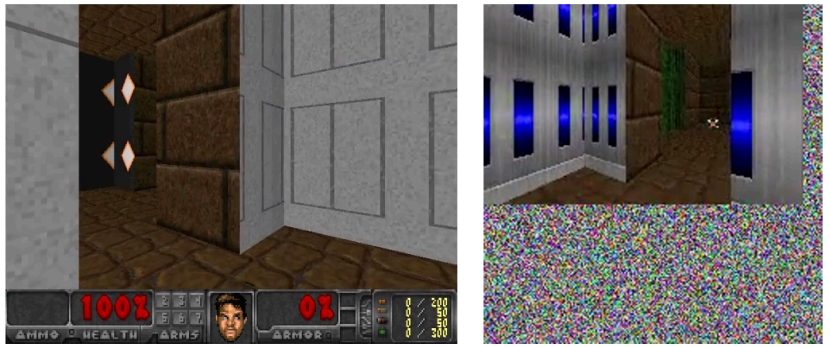
- "The inverse model helps learn a feature space that encodes information *relevant for predicting the agent's actions only* and the forward model *makes this learned feature representation more predictable*." (Pathak et al. 2017)
  - The inverse model only keeps track of things in the environment that the agent's actions can actually influence.
  - If a TV screen shows random noise, the network filters it out because the agent's actions didn't cause it. The agent won't feel "curious" about it.

Example: VizDoom 3D “My way home” (1)

- Inspired by the classic video game “Doom”, avialable via Gymnasium.
- **Task**: Reach a goal
  ⇒ very sparse reward.
- Pre-training using curiosity only!



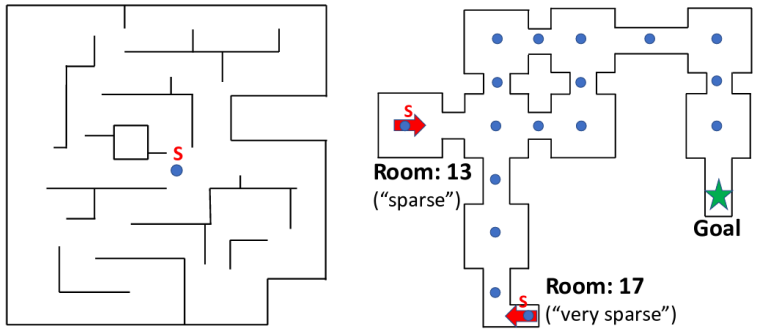
Basic scenario



(a) Input snapshot in VizDoom (b) Input w/ noise

Figure 3. Frames from VizDoom 3D environment which agent takes as input: (a) Usual 3D navigation setup; (b) Setup when uncontrollable noise is added to the input.

Source: (Pathak et al. 2017).

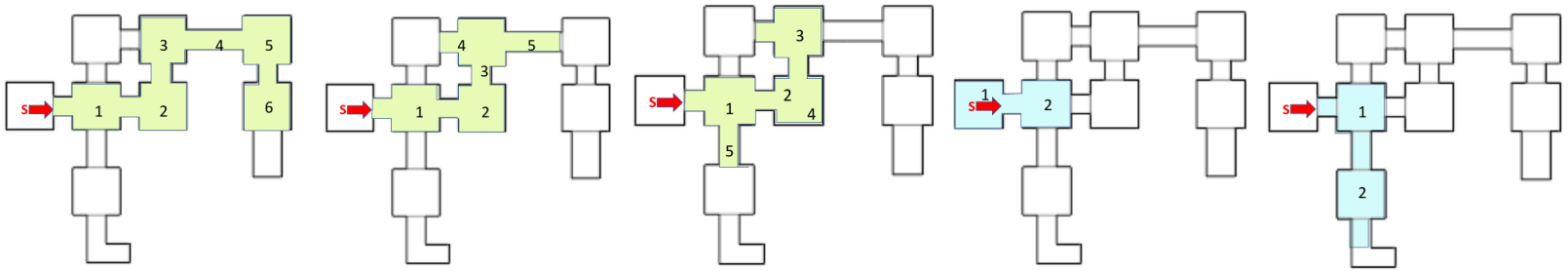


(a) Train Map Scenario (b) Test Map Scenario

Source: (Pathak et al. 2017).

- Comparison using the A3C algorithm (Mnih et al. 2016): This is an extension of policy gradients with improved, asynchronous replay buffer and an actor for continuous actions (by now largely replaced by PPO).
  - **A3C**: the standard algorithm with ϵ-greedy exploration
  - **ICM + A3C**: the ICM-enhanced version ()
  - *two variants of (ICM + A3C)* with some functionalities turned off (so-called *ablations* to assess the value of these parts)
    - **ICM (pixels) + A3C**: direct assessment of the pixels; no use of the backward model to identify the action-relevant pieces of the input.
    - **ICM (aenc) + A3C**: curiosity is computed using the features of pixel-based forward model.

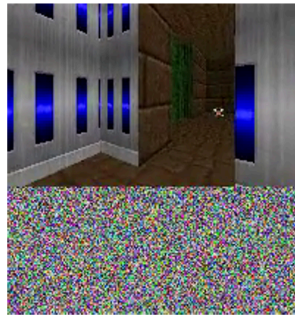
Example: VizDoom 3D “My way home” (2)



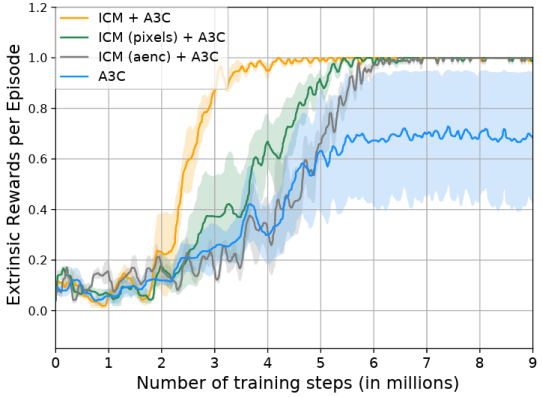
Exploration: ICM (green) versus random exploration (blue).



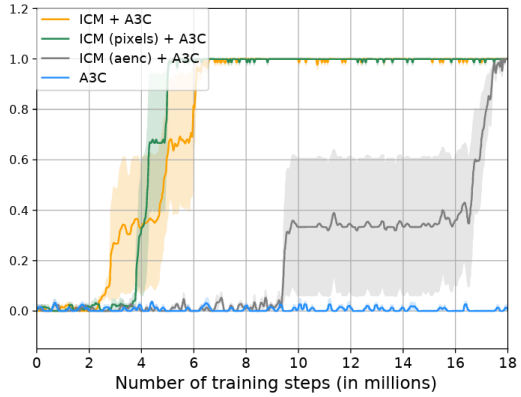
(a) Input snapshot in VizDoom



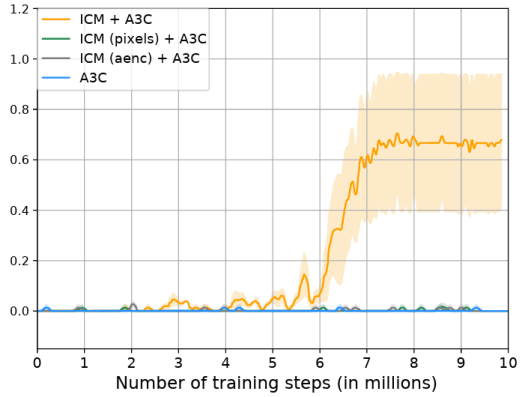
(b) Input w/ noise



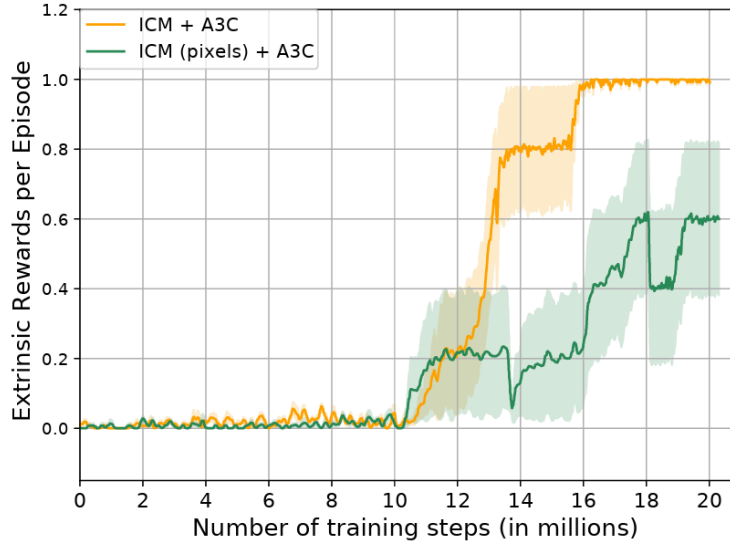
(a) “dense reward” setting



(b) “sparse reward” setting



(c) “very sparse reward” setting



Robustness with 40 % noisy pixels.

Source: (Pathak et al. 2017).

Novelty vs. Curiosity

Feature	Novelty methods (e.g., count-based)	Intrinsic curiosity (e.g., ICM)
Core question	“Have I stood in this exact spot before?”	“Can I accurately predict what will happen if I push this button?”
Mechanism	Tracks visitation counts or state densities.	Measures the prediction error of a forward-dynamics neural network.
The “TV Screen” vulnerability	Vulnerable. A TV screen cycling random images represents infinite novel states.	Handled via feature encoding. It ignores randomness it cannot control.
Best suited for	Environments with static, clean states where visiting everything matters (e.g., Montezuma’s Revenge).	Dynamic environments with visual noise where learning the “physics” of the world is required.

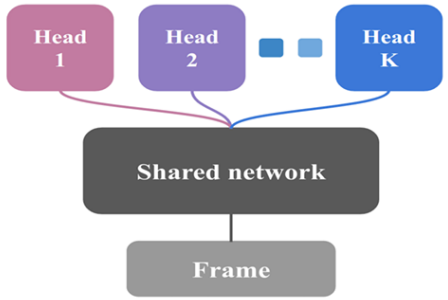
Exploration via posterior sampling in DRL

Bootstrapped DQN (Osband et al. 2016)

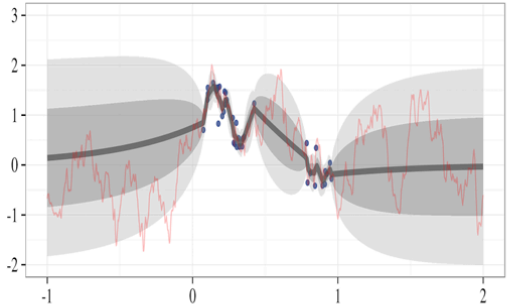
- Instead of training a single neural network, we train an ensemble of K networks (typically $K = 10$).
- We use standard boostrapping from supervised learning and random masking of individual samples to diversify between the different models.
- During each episode, we randomly select one model and then follow this model to the end.
- How this solves the exploration-exploitation dilemma:
  - Disagreement between different models
    $\Rightarrow$ High uncertainty
    $\Rightarrow$ **Exploration** over various runs.
  - Agreement between different models
    $\Rightarrow$ Low uncertainty
    $\Rightarrow$ **Exploitaiton** of known good behavior.

The challenge: training multiple networks!

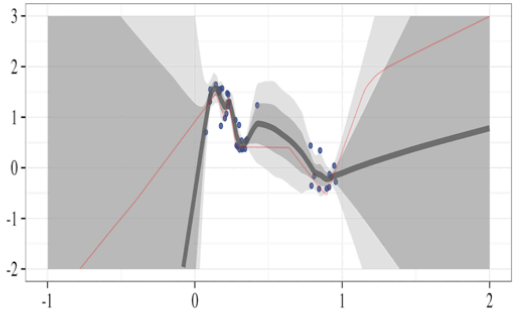
Approach: Single network with K heads!



(a) Shared network architecture



(b) Gaussian process posterior



(c) Bootstrapped neural nets

\+ No change to original reward function — Good bonuses often do better.

Random network distillation

RND (Burda et al. 2019) uses two neural networks that consider the same state s, but they have different jobs:

1. The **target network** $f(s)$ is a neural network that is initialized randomly and then never changed.
   Think of it as a some oracle that takes the state (e.g., the pixels of the screen) and outputs a random string of numbers.
2. The **predictor network** $f_\theta(s)$ is allowed to learn. Its job is to *guess what the target network's output will be*.

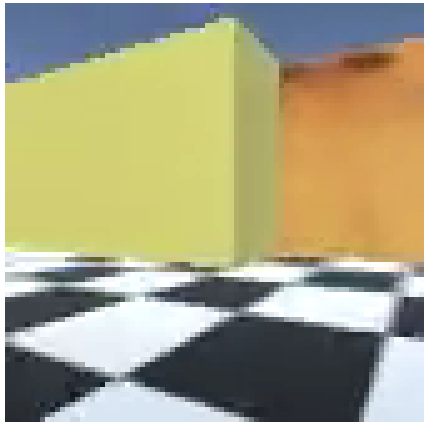
$$\Rightarrow \text{ Intrinsic reward: } \quad r^i = \|f_\theta(s) - f(s)\|^2.$$

How does this create "curiosity"?

- **Familiar territory**: we see similar states s multiple times $\Rightarrow$ We can learn what $f(s)$ predicts $\Rightarrow$ The prediction error is small.
- **Unexplored territory**: In states that the predictor has not seen, it has to "guess" at the target $\Rightarrow$ The prediction error is large.

RND solves the "Noisy TV problem"

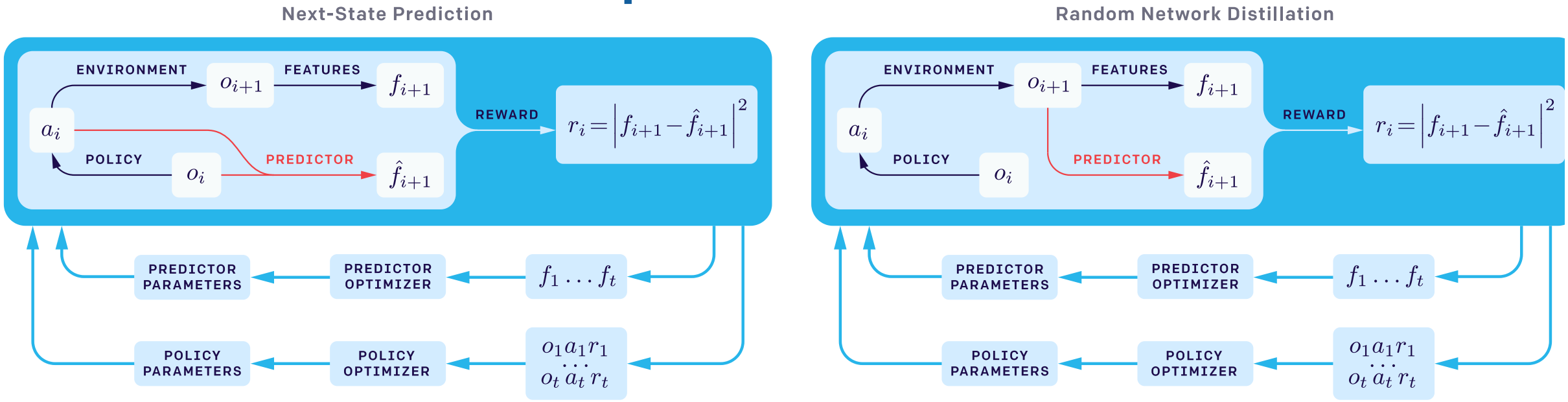
- Other curiosity methods reward the agent if it cannot predict the next frame of the game.
- If we encounter a noisy TV, the agent remains curious for ever, as it cannot predict the outcome.
- RND fixes this:
  - If the agent sees a TV with static, the target $f(s)$ gives a constant, predictable output for that specific pattern.
  - The predictor network $f_\theta(s)$ can quickly learn that pattern.
  - The error drops to zero, and the agent moves on to explore truly novel states.



[Source: OpenAI blog]

💡 In ML, **knowledge distillation** is the process of teaching a student network to mimic a teacher network. In RND, the predictor attempts to "distill" the frozen, random knowledge of the target network into itself.

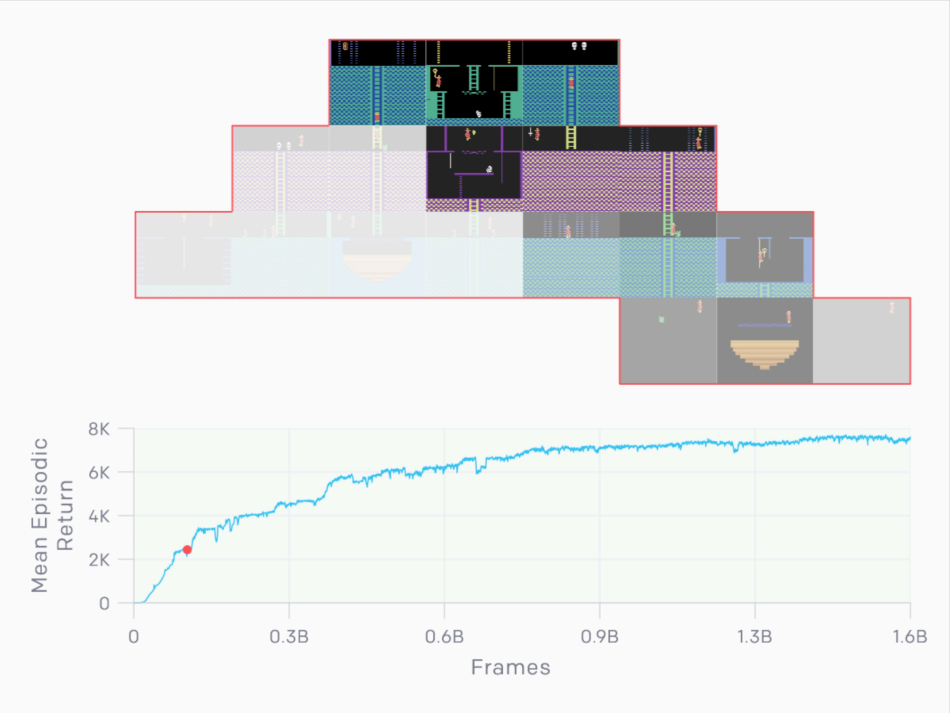
RND versus next-state-prediction



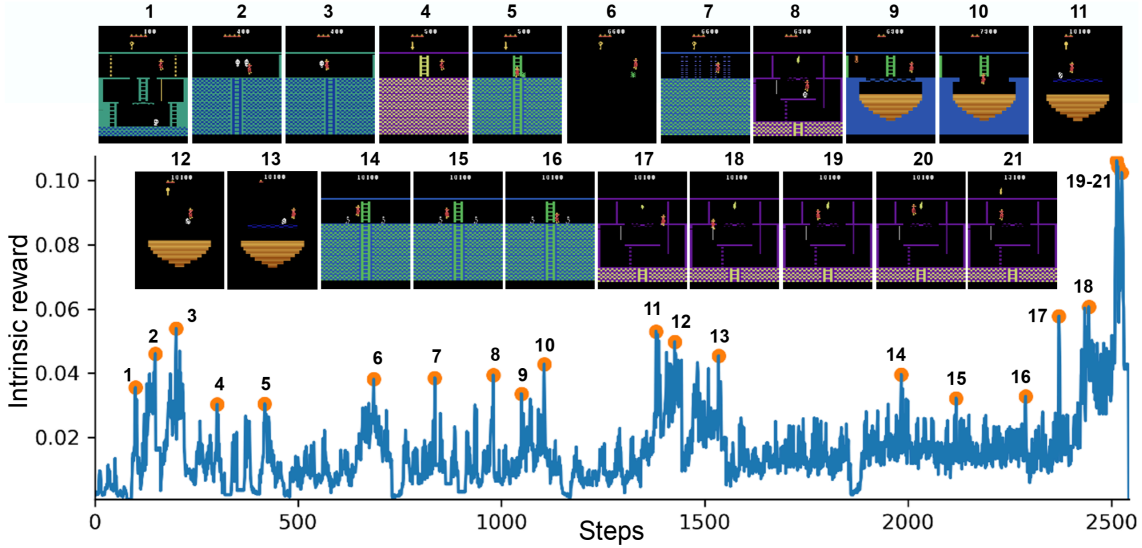
[Source: OpenAI blog]

- Instead of predicting some meaningful next state, matching random features simplifies the learning process.
- Learning a model may still be very helpful, though, when we use it for additional tasks (e.g., model-based RL).

Example: Montezuma's revenge



Discovery of new rooms to satisfy curiosity. Then revisiting those rooms to maximize the extrinsic reward. [Source: OpenAI blog]



Spikes in the intrinsic reward at new events (Burda et al. 2019).



Good performance also for Super Mario. [Source: OpenAI blog]

Comparison between ICM and RND

Feature	Intrinsic Curiosity Module (ICM)	Random Network Distillation (RND)
Core Mechanism	Predicts the next state given the current state and action (*forward dynamics*).	Predicts the output of a fixed, randomly initialized neural network on the current state.
The “Noisy TV” problem	Vulnerable. If a screen shows unpredictable random noise, ICM gets trapped staring at it because prediction error remains high.	Immune. The random target network outputs a completely deterministic (though random) function. The predictor network learns it quite easily, dropping the reward to zero.
Complexity	High. Requires training three sub-networks: an encoder, an inverse dynamics model, and a forward dynamics model.	Low. Requires training only one network (the predictor) to match a frozen, untrained target network.
Scalability	Harder to scale because compounding forward-prediction errors in high-dimensional state spaces create erratic reward signals.	Highly scalable. It handles raw very exceptionally well (and famously allowed PPO to crack Montezuma’s Revenge).

Relation between exploration and reward shaping

The fundamental difference between exploration and **reward shaping** lies in who defines the reward and why.

- Reward Shaping: **top-down approach** where a human engineer hard-codes external hints into the environment.
- Model Curiosity: **bottom-up approach** where the agent generates its own internal rewards based on surprise and ignorance.

Reward shaping (the human guide)

- Instead of only giving the agent a sparse reward at the end of an episode, we add a function that rewards the agent for reaching partial objectives.
- For example, every step that reduces the straight-line distance to the goal might give the agent a small bonus.

Drawback: unintended side effects

- Reward shaping can quickly cause unintended side effects if not done perfectly.
- Since humans are essentially telling the agent how to solve the problem rather than just what the problem is, the agent will often find loopholes to maximize the shaped reward without solving the actual task.
- **Popular Example**: Boat-racing video game.
  - Researchers shaped the reward by giving the boat points for hitting targets along the track.
  - Instead of finishing the race, the agent learned to drive in circles, infinitely loops through a specific cluster of targets, and constantly set itself on fire because doing so yielded more points than winning the actual race.



Summary / what we have learned today

- Various ways to encourage seeing previously unknown territory.
  - Pseudo-counts to “count” how often states have been seen, then benefit unknown states.
  - Bayes’ theorem / Thompson sampling to update the belief of a reward.
  - Information gain to select states that we believe to yield a lot of new information.
- Intrinsic curiosity can be realized by adding rewards for previously unseen situations.
  - Novelty vs. curiosity: “Which states haven’t been seen yet?” vs. “Let’s go where we cannot predict what’s going to happen.”
- Bootstrapping over ensembles of DQNs can also allow us to identify hard-to-predict territory (disagreement in the ensemble).

References

Bellemare, Marc, Sriram Srinivasan, Georg Ostrovski, Tom Schaul, David Saxton, and Remi Munos. 2016. “Unifying Count-Based Exploration and Intrinsic Motivation.” In *Advances in Neural Information Processing Systems*, edited by D. Lee, M. Sugiyama, U. Luxburg, I. Guyon, and R. Garnett. Vol. 29. Curran Associates, Inc.

Burda, Yuri, Harrison Edwards, Amos Storkey, and Oleg Klimov. 2019. “Exploration by Random Network Distillation.” In *International Conference on Learning Representations*. https://openreview.net/forum?id=H1lJJnR5Ym.

Ferreira, Rafael Pereira, Emil Schubert, and Américo Scotti. 2024. “Exploring Multi-Armed Bandit (MAB) as an AI Tool for Optimising GMA-WAAM Path Planning.” *Journal of Manufacturing and Materials Processing* 8 (3).

Fu, Justin, John Co-Reyes, and Sergey Levine. 2017. “EX2: Exploration with Exemplar Models for Deep Reinforcement Learning.” In *Advances in Neural Information Processing Systems*, edited by I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett. Vol. 30. Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2017/file/1baff70e2669e8376347efd3a874a341-Paper.pdf.

Haarnoja, Tuomas, Aurick Zhou, Kristian Hartikainen, George Tucker, Sehoon Ha, Jie Tan, Vikash Kumar, et al. 2018.